

Inkrementelle Kompilierung

Auf den Spuren des Typst-Compilers

Tristan Pieper — tristan.pieper@uni-rostock.de

Dienstag, 16.06.2026

[1]





```
784 not in word) produces a landscape-orientation formatted ar
785 a wide left margin. Three environments are available for use
786 ``\verb[sigchi-a]'' template style, and produce formatted output in
787 the margin:
788 \begin{description}
789 \item[\texttt{[sidebar]}:] Place formatted text in the margin.
790 \item[\texttt{[marginfigure]}:] Place a figure in the margin.
791 \item[\texttt{[marginable]}:] Place a table in the margin.
792 \end{description}
793
794
795
796
797
798
799 \begin{acks}
800 To Robert, for the bagels and explaining CMX and color spaces.
801 \end{acks}
802
803
804
805
806 \bibliographystyle{ACM-Reference-Format}
807 \bibliography{sample-base}
808
809
810
811
812 \appendix
813
814 \section{Research Methods}
815
816 \subsection{Part One}
817
818
819 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi
820 malesuada, quam in pulvinar varius, metus nunc fermentum urna, id
821 sollicitudin purus odio sit amet enim. Aliquam ullamcorper eu ipsum
822 vel mollis. curabitur quis dictum nisl. Phasellus vel semper risus, et
823 lacinia dolor. Integer ultricies commodo sem nec semper.
824
825
826
827
828
829
830
831
832 \subsection{Part Two}
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

The Name of the Title Is Hope

Ben Trivato* G.K.M. Tobus trivato@corporation.com webmaster@mayorville-ohio.com Institute for Clarity in Documentation Dublin, Ohio, USA	Lars Tharvold The Tharvold Group Helsinki, Iceland lars@tharvold.org	Valerie Branger Initia Paris-Rocquencourt Rocquencourt, France
Aparna Patel Rajiv Gandhi University Doomsday, Arunachal Pradesh, India	Huifen Chan Tsinghua University Haidian Qs, Beijing Shi, China	Charles Palmer Palmer Research Laboratories San Antonio, Texas, USA cpalmer@prl.com
John Smith The Tharvold Group Helsinki, Iceland john@tharvold.org	Julius P. Kumquat The Kumquat Consortium New York, USA jpkumquat@consortium.net	

Abstract
A clear and well-documented $\LaTeX$ document is presented as an article formatted for publication by ACM in a conference proceedings or journal publication. Based on the "acmart" document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work.

CCS Concepts • the ACM Use This Code • Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format
Ben Trivato, G.K.M. Tobus, Lars Tharvold, Valerie Branger, Aparna Patel, Huifen Chan, Charles Palmer, John Smith, and Julius P. Kumquat. 2018. The Name of the Title Is Hope. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 9 pages. <https://doi.org/XXXXXX/XXXXXX/XXXXX>

1 Introduction
ACM's consolidated article template, introduced in 2017, provides a consistent $\LaTeX$ style for use across ACM publications, and incorporates accessibility and metadata-extraction functionality necessary for future Digital Library endeavors. Numerous ACM and SIG-specific $\LaTeX$ templates have been examined, and their unique features incorporated into this single new template.

If you are new to publishing with ACM, this document is a valuable guide to the process of preparing your work for publication. If you have published with ACM before, this document provides insight and instruction into more recent changes to the article template.

The "acmart" document class can be used to prepare articles for any ACM publication – conference or journal, and for any stage of publication, from review to final "camera-ready" copy, to the author's own version, with very few changes to the source.

2 Template Overview
As noted in the introduction, the "acmart" document class can be used to prepare many different kinds of documentation – a double-anonymous initial submission of a full-length technical paper, a two-page SIGGRAPH Emerging Technologies abstract, a "camera-ready" journal article, a

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

Trivato et al.

Your document will be returned to you for revision if modifications are discovered.

4 Typefaces
The "acmart" document class requires the use of the "Libertine" typeface family. Your $\TeX$ installation should include this set of packages. Please do not substitute other typefaces.

2.1 Template Styles

Probleme mit L^AT_EX

1. 40 Jahre alte Syntax

Probleme mit L^AT_EX

1. 40 Jahre alte Syntax
2. kryptische Fehlermeldungen

Probleme mit L^AT_EX

1. 40 Jahre alte Syntax
2. kryptische Fehlermeldungen
3. langsame Compilezeit

Probleme mit L^AT_EX

1. 40 Jahre alte Syntax
2. kryptische Fehlermeldungen
3. langsame Compilezeit
4. uvm.

Probleme mit L^AT_EX

- | | | |
|-------------------------------|---|------------------|
| 1. 40 Jahre alte Syntax | } | subjektiv |
| 2. kryptische Fehlermeldungen | | |
| 3. langsame Compilezeit | } | objektiv messbar |
| 4. uvm. | | |

Typst. Eine Lösung?¹

```
1 _Hallo Welt._ Dies ist  
  ein *test*.
```

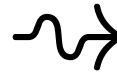


```
2
```

```
3 #table(columns: 3,
```

```
4     [a], [b], [c],
```

```
5     [d], [e], [f]))
```



Hallo Welt. Dies ist
ein **Test**.

a	b	c
d	e	f

¹Diese Folien sind in Typst erstellt.

Typst. Eine Lösung?

1. modernes Sprachdesign
2. verständliche Fehlermeldungen

Typst. Eine Lösung?

1. modernes Sprachdesign
2. verständliche Fehlermeldungen
3. schneller Compiler (Live-Preview)

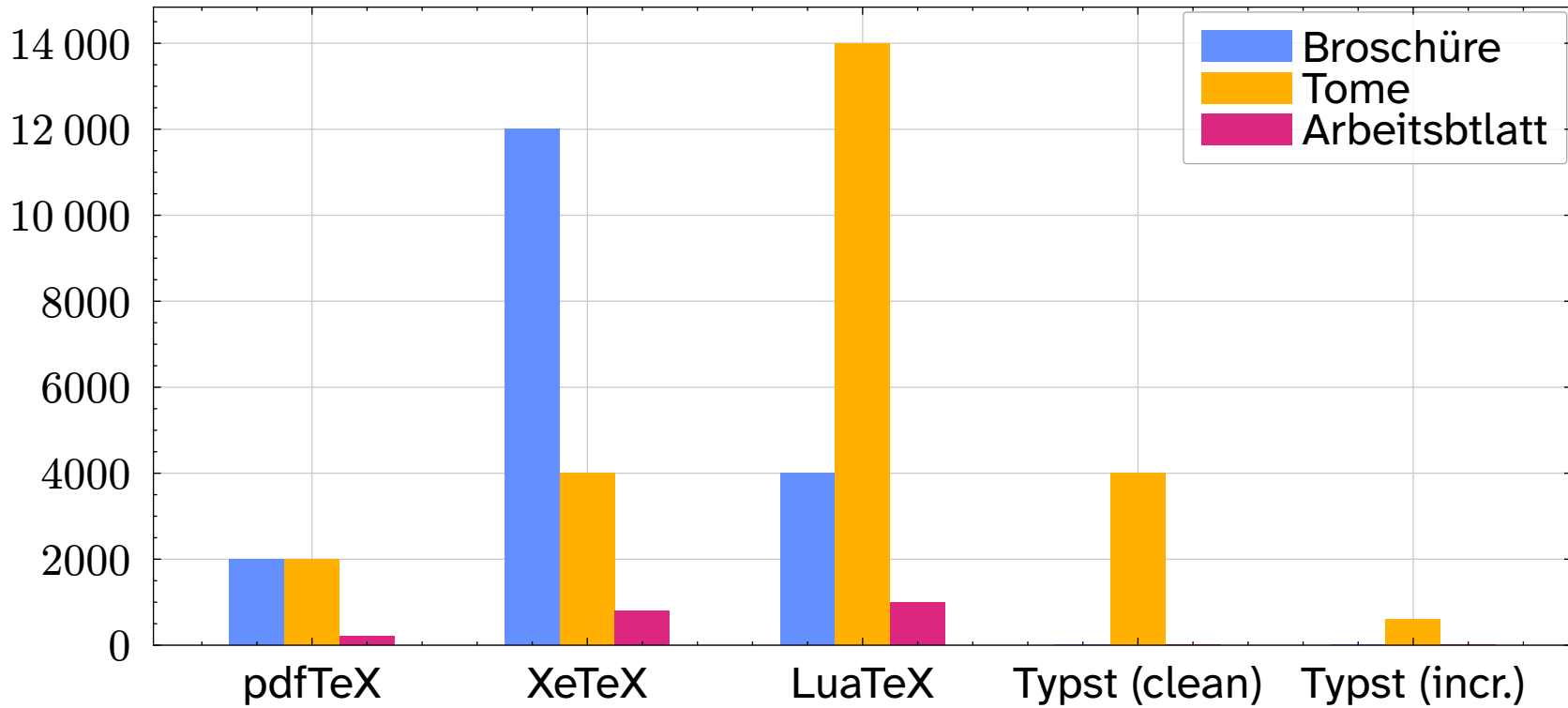


Abbildung 1: Benchmark des Typst-Compilers 2022, Angaben in ms [2, S. 70].

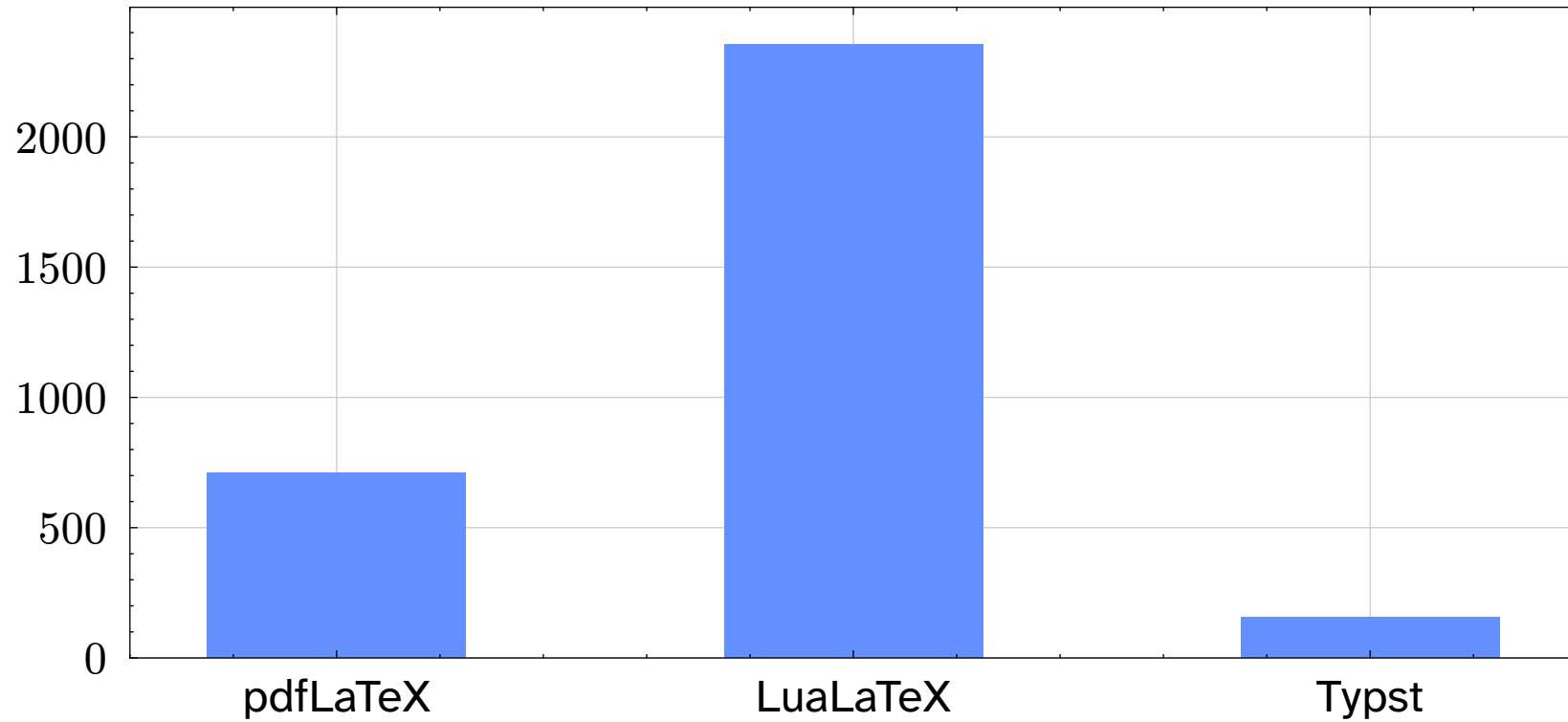


Abbildung 2: Benchmark des Typst-Compilers 2026, Angaben in ms [3].

Wie macht Typst das?

Funktionen in Typst

```
1 #let global = 3
2
3 #let increment() = {
4     global = global + 1
5 }
6
7 #increment()
```

A small teal square icon with a white lowercase letter 't' inside, representing the Typst logo.

Funktionen in Typst

```
1 #let global = 3
2
3 #let increment() = {
4     global = global + 1
5 }
6
7 #increment()
```



error: variables from outside
the function are read-only and
cannot be modified

```
test.typ:4:4
4      global = global + 1
      ^^^^^^^
```


Funktionen in Typst

Definition

Eine Funktion ist *wirkungsfrei* (ohne „side-effects“), gdw. sie den Zustand des Programmes nicht verändert.

Funktionen in Typst

Definition

Eine Funktion ist *wirkungsfrei* (ohne „side-effects“), gdw. sie den Zustand des Programmes nicht verändert.

Bsp.: alle mathematischen Funktionen sind wirkungsfrei

Inkrementelle Kompilierung

- jede Funktion in Typst ist $f : \text{Value}^n \rightarrow \text{Value}$

Inkrementelle Kompilierung

- jede Funktion in Typst ist $f : \text{Value}^n \rightarrow \text{Value}$
- Cache konstruieren:

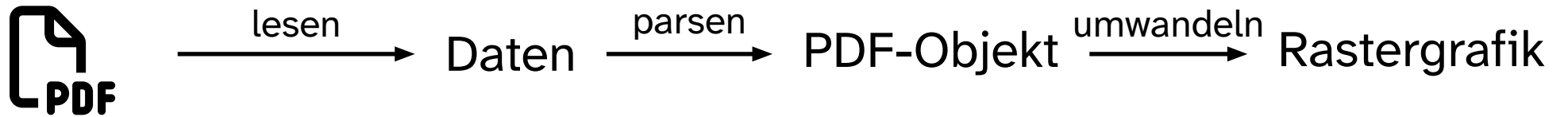
$\text{hash}(\langle f, v_1, v_2, \dots \rangle)$	v_{ret}
1011172c6...	[Hallo]
bad8c0dc4...	4
12fccd699...	table(columns:(auto,auto),chil...

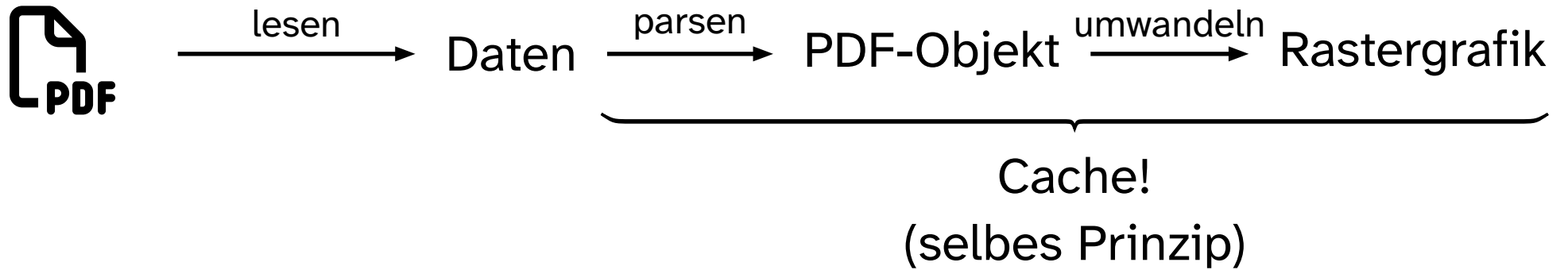
Inkrementelle Kompilierung

In main.typ:

```
1 #image("bookcover.pdf")
2 #include "chapters/chap01.typ"
3 #include "chapters/chap02.typ"
4 ...
```







```
1 impl PdfDocument {
```

```
2
```

```
3     pub fn new(data: Bytes) → PdfDocument {
```

```
4
```

```
    ...
```




```
1  impl PdfDocument {  
2      #[comemo::memoize]  
3      pub fn new(data: Bytes) → PdfDocument {  
4          ...  
}
```





```
1  
2 fn eval(sourceCode: &Source) → Value {  
3     ...  
4 }
```

```
1  #[comemo::memoize]
2  fn eval(sourceCode: &Source) → Value {
3      ...
4  }
```



```
1  #[comemo::memoize]
2  fn eval(sourceCode: &Source) → Value {
3      ...
4  }
```



```
1  #image("bookcover.pdf")
2  #include "chapters/chap01.typ"
3  #include "chapters/chap02.typ"
4  ...
```



```
1 #[comemo::memoize]
2 fn eval(sourceCode: &Source) → Value {
3     ...
4 }
```



```
1 #image("bookcover.pdf")
2 #include "chapters/chap01.typ"
3 #include "chapters/chap02.typ"
4 ...
```



⇒ im Cache nicht alle Abhängigkeiten berücksichtigt

```
1 #[comemo::memoize]
```



```
2 fn eval(sourceCode: &Source, fs: &Fs) → Value { ... }
```

```
1 #[comemo::memoize]
2 fn eval(sourceCode: &Source, fs: &Fs) → Value { ... }
```



↑ fs.load(path)

path	source
/chapters/chap01.typ	The palace still shook occasionally as ...
/chapters/chap02.typ	The Wheel of Time turns, and Ages ...
...	...

```
1  #[comemo::memoize]
2  fn eval(sourceCode: &Source, fs: Tracked<&Fs>) → Value { ... }
```




```
1 #[comemo::memoize]
2 fn eval(sourceCode: &Source, fs: Tracked<&Fs>) → Value { ... }
```



↓ fs.load(path)

	path	hash(v_{ret})
👁	/chapters/chap01.typ	03f200a3b8bb...
👁	/chapters/chap02.typ	2084ff93ce00...
	...	...

```
1 #[comemo::memoize]
2 fn eval(sourceCode: &Source, fs: Tracked<&Fs>) → Value { ... }
```



↓ fs.load(path)

	path	hash(v_{ret})
👁	/chapters/chap01.typ	03f200a3b8bb...
👁	/chapters/chap02.typ	2084ff93ce00...
	...	...

⇒ nur relevantes Verhalten von fs wird getrackt

```
1 #[comemo::memoize]
2 fn eval(sourceCode: &Source, fs: Tracked<&Fs>) → Value { ... }
```



↓ fs.load(path)

	path	hash(v_{ret})
👁	/chapters/chap01.typ	03f200a3b8bb...
👁	/chapters/chap02.typ	2084ff93ce00...
	...	...

⇒ nur relevantes Verhalten von fs wird getracked

⇒ getrackte Argumente bilden „Constraints“ für Cache-Eintrag

```

1  #[comemo::memoize]
2  fn eval(sourceCode: &Source, fs: Tracked<&Fs>) → Value { ... }

```



- Caching dann wie folgt:

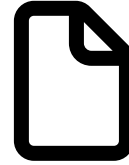
$\langle v_1, v_2, \dots, v_n \rangle$	Constraints		v_{ret}
...	path	hash(V_{ret})	...
	/chapters/chap01.typ	03f200a3b8bb...	
	/chapters/chap02.typ	2084ff93ce00...	



main.typ



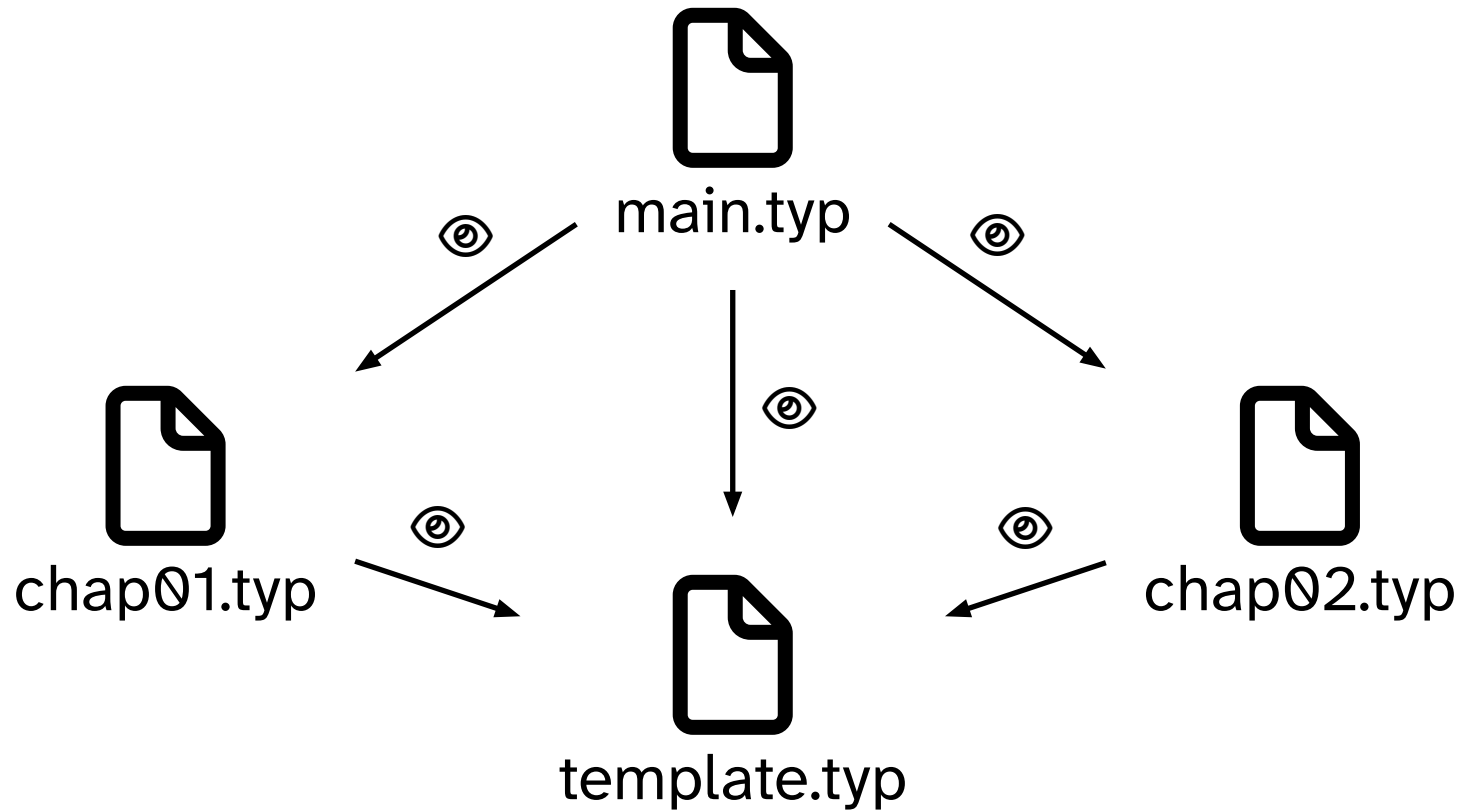
chap01.typ



chap02.typ



template.typ



Inkrementelle Kompilierung

- Wirkungsfreiheit beschreibt unabhängigen Code
- Typst nutzt Wirkungsfreiheit um Funktionen zu cachen
- Constraints ermöglichen effiziente Validierung des Caches

(Diese Folien wurde in 2,5 s und mit Caching in 150 ms kompiliert.)

Was ist mit (La)T_EX?

„Don tried very hard not to make TeX a programming language.
Unfortunately, he didn't succeed.“¹

— Michael Pass, Schüler von Donald Knuth, dem Erfinder von TeX

¹Norman Ramsey: Stackoverflow. <https://stackoverflow.com/questions/1931238/if-tex-is-a-programming-language-how-could-i-start-programming-in-tex>, letzter Zugriff: 1.6.2025, 11:39 Uhr.

Pure Functions/Macros möglich?

```
1 ...  
2 \def\A{A}  
3 "\A"  
4 ...
```

TeX

Pure Functions/Macros möglich?

```
1 ...  
2 \def\A{A}  
3 "\A"  
4 ...
```

TeX

„A“

Pure Functions/Macros möglich?

```
1  ...  
2  \def\A{A}  
3  \def\B{\def\A{C}}  
4  "\A \B \A"  
5  ...
```

TeX

Pure Functions/Macros möglich?

```
1  ...
2  \def\A{A}
3  \def\B{\def\A{C}}
4  "\A \B \A"
5  ...
```

TeX

„AC“

Pure Functions/Macros möglich?

```
1  ...  
2  \def\A{A}  
3  \def\B{\def\A{C}}  
4  "\A \B \A"  
5  ...
```

TeX

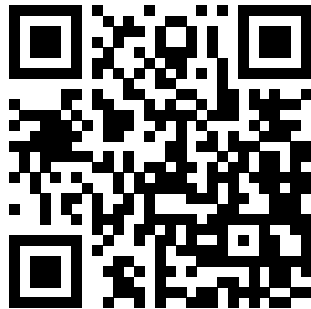
„AC“

⇒ \B nicht wirkungsfrei

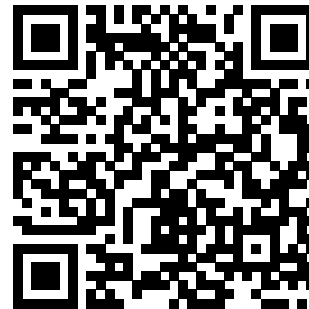
Inkrementelle Kompilierung

- Neukompilierung nur dessen, was sich ändert
- Rest mittels constrained Memoization speichern
- durch Wirkungsfreiheit des Typst-Codes und im Compilerdesign besonders effektiv

Typst:



Diese Folien:



Bibliografie

- [1] Pexels, „Adult, Annoyed, Blur image. Free for use.“. Zugegriffen: 3. Juni 2026. [Online]. Verfügbar unter: <https://pixabay.com/photos/adult-annoyed-blur-burnout-1850268/>
- [2] M. Haug, „Fast Typesetting with Incremental Compilation“, Masterarbeit, Berlin, 2022. Zugegriffen: 12. Mai 2026. [Online]. Verfügbar unter: <https://web.archive.org/web/20221022111033/https://www.user.tu-berlin.de/mhaug/fast-typesetting-incremental-compilation.pdf>
- [3] Speedata, „Speedata/typesetting-benchmark: Compare typesetting speed of SP, typst, weasyprint, FOP, latex and LuaLaTeX“. Zugegriffen: 31. Mai 2026. [Online]. Verfügbar unter: <https://github.com/speedata/typesetting-benchmark>
- [4] L. Mädje, „Typst. A Programmable Markup Language for Typesetting“, Masterarbeit, Berlin, 2022. Zugegriffen: 12. Mai 2026. [Online]. Verfügbar unter: <https://laurmaedje.github.io/programmable-markup-language-for-typesetting.pdf>
- [5] L. Mädje, „Frozen State“. Zugegriffen: 12. Mai 2026. [Online]. Verfügbar unter: <https://laurmaedje.github.io/posts/frozen-state/>
- [6] L. Mädje, „Types and Context“. Zugegriffen: 12. Mai 2026. [Online]. Verfügbar unter: <https://laurmaedje.github.io/posts/types-and-context/>
- [7] L. Mädje, „What If LaTeX Had Instant Preview?“. Zugegriffen: 12. Mai 2026. [Online]. Verfügbar unter: <https://laurmaedje.github.io/posts/comemo/>
- [8] L. Mädje, „TeX and Typst: Layout Models“. Zugegriffen: 12. Mai 2026. [Online]. Verfügbar unter: <https://laurmaedje.github.io/posts/layout-models/>
- [9] D. Salomon, *The Advanced TEXbook*. New York, NY: Springer New York, 1995.
- [10] L. Mädje, „Designing for incrementality“. Zugegriffen: 31. Mai 2026. [Online]. Verfügbar unter: <https://www.youtube.com/live/6LTwtYuj69M>

Weitere Typst-Beispiele

Theorem 1.1. $f(n) = 7n^4 - 4n^3 + \lceil n^e + \pi \rceil$

Es gilt $f \in \Theta(n^4)$.

Beweis:

1. $\Theta(n^4) := O(n^4) \cap \Omega(n^4)$, d.h. $f \in \Theta(n^4) \leftrightarrow (f \in O(n^4) \wedge f \in \Omega(n^4))$.

2. $f \in O(n^4)$?

a) Ja, denn:

$$|f(n)| = |a_0 + a_1n + a_2n^2 + \dots + a_mn^m| \leq \left(\sum_{i=0}^m \frac{|a_i|}{n^{m-i}} \right) n^m \leq \left(\sum_{i=0}^m |a_i| \right) n^m$$

$$c = \left(\sum_{i=0}^m |a_i| \right)$$

b) Mit gewähltem c gilt, dass f nicht schneller wächst als n^4 .

3. $f \in \Omega(n^4)$? D.h. $f \in \Omega(n^4) \leftrightarrow g(n) = n^4 \in O(f)$. Also $g \in O(f)$? Siehe dazu Schritt 2: c kann in Bezug auf g ebenfalls gewählt so werden, dass $g \in O(f)$.

4. Daher $f \in O(n^4) \wedge f \in \Omega(n^4)$, also $f \in \Theta(n^4)$.

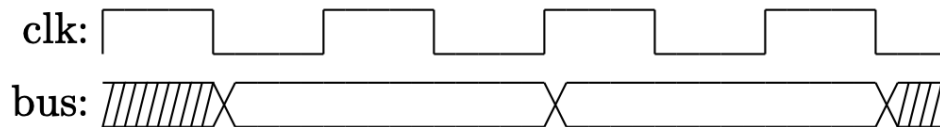
□

$$A^\top, \boldsymbol{\nabla} \times \boldsymbol{E} = -\frac{\partial \boldsymbol{B}}{\partial t}, \quad \Lambda^\mu{}_\nu = \begin{pmatrix} 1 & \\ & \mathbb{R} \end{pmatrix}, \quad f(x,y) \, \mathrm{d}x \, \mathrm{d}y, \quad \mathrm{d}^3 \boldsymbol{x} \, \mathrm{d}y, \quad \Delta^2 x \wedge \Delta^2 y, \quad \frac{\mathrm{D}\varphi}{\mathrm{D}t} = \frac{\partial \varphi}{\partial t} + \boldsymbol{u} \boldsymbol{\nabla} \varphi$$

$$H(f)=\begin{bmatrix}\frac{\partial^2 f}{\partial x^2}&\frac{\partial^2 f}{\partial x\partial y}\\\frac{\partial^2 f}{\partial y\partial x}&\frac{\partial^2 f}{\partial y^2}\end{bmatrix},\quad v^a=\sum_{i=1}^n\alpha_i\hat{u}^i,\quad\left\{(x,y)\left|\frac{\partial^3 f}{\partial x^2\partial y}+\frac{\partial^3 f}{\partial x\partial y^2}<\varepsilon\right.\right\}$$

$$-\frac{1}{c^2}\frac{\partial^2}{\partial t^2}\psi+\nabla^2\psi=\frac{m^2c^2}{\hbar^2}\psi,\quad |n^{(1)}\rangle=\sum_{k\notin D}\frac{\langle k^{(0)}|V|n^{(0)}\rangle}{E_n^{(0)}-E_k^{(0)}}|k^{(0)}\rangle,\quad \int_V\mathrm{d}V\left(\frac{\partial\mathcal{L}}{\partial\varphi}-\partial_\mu\left(\frac{\partial\mathcal{L}}{\partial(\partial_\mu\varphi)}\right)\right)=0$$

$$\mathrm{d}^2s=-\left(1-\frac{2GM}{r}\right)\mathrm{d}^2t+\left(1-\frac{2GM}{r}\right)^{-1}\mathrm{d}^2r+r^2\,\mathrm{d}^2\Omega$$



Übung 9

Aufgabe 1 — Diagonalsprache der RAM

- (i) Seien für diese Teilaufgabe $\ulcorner \alpha \urcorner$ objektsprachliche Ausdrücke mit $\alpha \in \Sigma^+$. Sei $w \circ v = wv$, dementsprechend $\ulcorner a \urcorner \circ \ulcorner b \urcorner = \ulcorner ab \urcorner$.

Sei für alle RAMs R mit der Befehlsmenge B eine Gödelnummer definiert als $G(R) = \langle R \rangle, \langle R \rangle \in \{\ulcorner 0 \urcorner, \ulcorner 1 \urcorner\}^*$.

Dafür sei $\text{bin} : \mathbb{N} \rightarrow \{\ulcorner 0 \urcorner, \ulcorner 1 \urcorner\}^+$ die Binärdarstellung einer Zahl $x \in \mathbb{N}$. Zusätzlich sei $\text{double} : \Sigma^+ \rightarrow \Sigma^+$ diejenige Funktion, die jedes Symbol eines Wortes verdoppelt:

$$\text{double}(x) = \begin{cases} x \circ x & \text{wenn } x \in \Sigma \\ \text{double}(v) \circ \text{double}(w) & \text{wenn } x = vw \text{ und } v, w \in \Sigma^+ \end{cases}$$

Sei $G_B : B \rightarrow \{\ulcorner 0 \urcorner, \ulcorner 1 \urcorner\}^*$ wie folgt definiert:

$$G_B(x) = \begin{cases} \ulcorner 0 \urcorner & \text{wenn } x = \text{LOAD} \\ \ulcorner 1 \urcorner & \text{wenn } x = \text{STORE} \\ \ulcorner 10 \urcorner & \text{wenn } x = \text{ADD} \\ \ulcorner 11 \urcorner & \text{wenn } x = \text{SUB} \\ \dots & \\ \text{bin}(n) & \text{wenn } x = B, n \leq |B|, n \in \mathbb{N} \end{cases}$$

Definition 2.13 (*lebendige Transition*). Eine Transition t ist lebendig bei m , gdw. $\forall m' \in R_N(m) : \exists m'' \in R_N(m') : \text{aktiviert}(t)$. Also dass für alle $m' \in R_N(m)$ eine erreichbare Markierung m'' existiert, in der t aktiviert ist.

Definition 2.14 (*k-beschränkte Stelle*). Eine Stelle heißt k -beschränkt bei m , gdw. in allen $m' \in R_N(m)$ gilt, dass $m'(s) \leq k$.

Definition 2.15 (*sichere Stelle*). Eine Stelle s heißt sicher, gdw. sie 1-beschränkt ist.

Definition 2.16 (*Homestate*). Eine Markierung m wird Homestate eines Petrinetzes (S, T, F, W, m_0) genannt, gdw. $\forall m' \in R_N(m_0) : m' \rightarrow m$. Also wenn m von allen erreichbaren Markierungen erreichbar ist.

2.3. NETZEIGENSCHAFTEN

Für ein Netz $N = (S, T, F, W, m_0)$ gelten die folgenden Eigenschaften.

Definition 2.17 (*Verklemmungsfreiheit*). Ein Netz N ist verklemmungsfrei, gdw. in jedem $m \in R_N(m_0)$ mindestens eine Transition aktiviert ist.

Definition 2.18 (*lebendiges Netz*). N heißt lebendig, gdw. alle Transitionen in N lebendig sind.

Theorem 2.1 Wenn N lebendig ist mit $|T| \geq 1$, dann ist N verklemmungsfrei.

Beweis. Angenommen N ist lebendig und es gibt mindestens eine Transition t . Wäre N nicht verklemmungsfrei, dann gäbe es eine Markierung m , in der keine Transition aktiviert ist, auch nicht t . Von dort aus ist keine Markierung m' erreichbar, in der t aktiviert ist. Damit wäre t nicht lebendig. Das ist ein Widerspruch. $\therefore$ Also ist das Theorem wahr. $\blacksquare$

Definition 2.19 (*Reversibilität*). N ist reversibel, gdw. $\forall m \in R_N(m_0) : m \rightarrow^* m_0$.

Theorem 2.2 Wenn N reversibel ist, dann ist jede Transition bei m_0 entweder tot oder lebendig.

Beweis. Wenn t in m_0 nicht tot ist, dann gibt es ein m' in dem t aktiviert ist, mit $m_0 \rightarrow^* m'$. Da N aber reversibel ist, gilt $\forall m \in R_N(m) : m \rightarrow^* m_0$. Also auch $\forall m \in R_N(m) : m \rightarrow m'$. Das ist gerade die Definition von Lebendigkeit. Wenn also eine Transition t nicht tot ist und N reversibel, dann ist t lebendig. Da eine Transition nicht sowohl lebendig als auch tot sein kann, ist jede Transition in einem reversiblen Netz entweder tot oder lebendig. $\blacksquare$

Theorem 2.3 N ist reversibel, gdw. m_0 ein Homestate ist.

Beweis. N ist reversibel. m_0 aus jeder Markierung $R_N(m_0)$ erreichbar. Das ist die Definition eines Homestates. Die Rückrichtung funktioniert analog. $\blacksquare$

Theorem 2.4 N ist reversibel, gdw. alle erreichbaren Zustände Homestates sind.

Beweis. Für alle Zustände gilt $m \rightarrow^* m_0 \rightarrow^* m'$ für $m, m' \in R_N(m_0)$. Damit ist jeder Zustand aus dem erreichbar und damit ein jeder ein Homestate. $\blacksquare$

3. ZUSTANDSRAUMANALYSE

3.1. ERREICHBARKEITSGRAPH

Definition 3.1 (*stark Zusammenhängende Komponente (SZK)*). Eine SZK ist ein gerichteter Teilgraph $G_{SZK} \subseteq G$, sodass es für jede Knoten $v, v' \in G_{SZK}$ einen Weg zwischen diesen gibt. Es gilt die Äquivalenzrelation $v \sim_{SZK} v'$.

Definition 3.2 (*terminale SZK (TSZK)*). Eine terminale SZK ist eine SZK, wobei aus keinem $v \in V_{SZK}$ ein Weg zu einem anderen $v' \notin V_{SZK}$ führt.

Im Folgenden ist der Graph, den wir analysieren wollen, der *Erreichbarkeitsgraph*, d.i. der Graph an Markierungen.

Definition 3.3 (*Erreichbarkeitsgraph*). Der Erreichbarkeitsgraph eines Petrinetzes N ist ein beschrifteter Graph (V', E) , wobei $V' = R_N(m_0)$ und $(m, t, m') \in E \iff m \rightarrow^t m'$.



Abbildung 3.1: Ein einfaches Petrinetz mit Erreichbarkeitsgraph.

Man erkennt im Erreichbarkeitsgraphen, dass dieser eine SZK bildet. Da die Marke immer im Kreis wandert und sich das Petrinetz nicht stark verändert, sieht auch der Erreichbarkeitsgraph entsprechend aus.

Tiefensuche zur SZK-Bestimmung:

- kleinste dfs-Zahl ist Wurzel der SZK (Was heißt das?)

Zur Entdeckung von SZK und TSZK kann man die Tiefensuche verwenden. Aus dem Tiefensuchbaum können wir die einzelnen Klassifizieren und mit bestimmten Zusammenhängen SZK und TSZK identifizieren. Zuerst zu den SZK. Je nach Anwendung des Tiefensuchalgorithmus, kann ein unterschiedliches Ergebnis rauskommen. Daher kann es für jedes Beispiel hier unterschiedliche Tiefensuchbäume geben. Das macht uns die Eigenschaften, die wir suchen wollen, jedoch nicht kaputt.

Definition 3.4 (*Kantenarten im Tiefensuchbaum*). In einem gerichteten Graphen G ist die Kante $e = (v, v') \in E_G$ mit Tiefensuchbaum T , ...

- eine Baumkante, falls $e \in E_T$.
- eine Vorwärtskante, falls es einen Weg zwischen v und v' in T gibt, aber e keine Baumkante ist.
- eine Rückwärtskante, falls es einen Weg zwischen v' und v in T gibt.
- eine Querkante, gdw. es sonst zu keiner Kantenart gehört.

Theorem 3.1 (*Kanteneigenschaften im Tiefensuchbaum*). Ist $(v, v') \in E_T, \forall x \in V_T : x.dfs$ die Tiefensuchnummer und handelt es sich dabei um eine ...

- Querkante, dann ist $v.dfs > v'.dfs$.



Abbildung 3.1: Die roten Knoten bilden eine stark zusammenhängende Komponente.

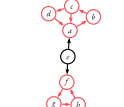


Abbildung 3.2: Die beiden roten Teilgraphen sind jeweils voneinander unabhängig. TSZK, t ist eine terminale SZK, die jedoch nicht terminiert, da sie in die anderen beiden überfließen kann. Sobald man aber in einer der roten TSZK ist, kann man diese nicht mehr verlassen.

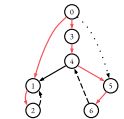


Abbildung 3.4: Die roten Knoten sind die des Suchbaums T . Gegenüber: Kanten sind Vorwärtskanten, gestrichelte sind Rückwärtskanten und die durchgezogene schwarze Kante ist eine Querkante.